AGRITRACE

DECENRALIZED SUPPLY CHAIN LEDGER & AI SYSTEMS MANUAL

This documentation covers the design, technical implementation, and comprehensive testing manual for the AgriTrace ecosystem. It detail the integration of Polygon blockchain, LangGraph RAG architectures, OpenCV/ViT diagnostics, Leaflet geospatial overlays, and multi-factor recovery.

Author: AgriTrace Core Systems

Version: 2.0 (PostgreSQL Migration)

Date: July 2026

Target Audience: Engineering and Quality Assurance Teams

1. Architectural Overview & Key Differentiators

AgriTrace solves a critical bottleneck in traditional supply chains: the trust deficit between producers and consumers. In legacy systems, crop parameters are stored in mutable centralized databases, leaving records vulnerable to unauthorized changes. AgriTrace bridges this gap by marrying cryptographic proofs with AI-powered physical verification.

Key Differentiators:

- **Dual-Ledger Hybrid Auditing:** Instead of storing all transactions on-chain (which is cost-prohibitive), AgriTrace hashes metadata off-chain (PostgreSQL) and writes the resulting cryptographic SHA-256 signatures to the Polygon blockchain smart contract. A mismatch instantly flags a batch as TAMPERED.
- **Geospatial Boundaries Privacy Control:** Leverages Leaflet interactive mapping to enforce privacy controls. Farmers and administrators can view exact boundaries, while buyers only see a fuzzed village/district overlay area.
- **AgriShield Automated Compliance:** Implements real-time seasonal compliance checking and Gemini-powered crop-to-image matching to detect fraud. Suspended farmers are instantly restricted to support chat channels.
- **Multi-Factor Password Recovery:** Features standard email OTP login alongside a local security question challenge recovery path.

2. AI Intelligence: LangGraph & LangChain Orchestration

The AgriTrace AI engine is built on LangGraph, enabling an event-driven multi-agent routing architecture. Rather than using a single monolithic prompt, user queries are routed dynamically by a Supervisor Agent to specialized sub-agents based on context.

LangGraph Sub-Agents and Roles:

1. **Supervisor Router:** Classifies user input intent and delegates state variables to child agents.
2. **Market Intelligence Agent:** Predicts agricultural crop demand, identifies price strategies, and recommends optimal timing by querying localized DB supply-demand snapshots.
3. **Weather Coordinator Agent:** Pulls real-time meteorological metrics from Tomorrow.io and OpenWeather API connectors to suggest cropping strategies.
4. **RAG Document Specialist:** Queries vectorized text indices (FAISS store + sentence-transformers) generated from uploaded agricultural guidelines manuals to answer complex farming questions in English, Hindi, and Kannada.

RAG Engine Code Pipeline:

```
def query_vector_store(query_text, k=3): embeddings =
SentenceTransformer('all-MiniLM-L6-v2') query_vector = embeddings.encode([query_text])
distances, indices = faiss_index.search(query_vector, k) chunks = [doc_store[idx] for idx
in indices[0]] return chunks
```

3. Computer Vision & Quality Assurance Pipeline

The computer vision pipeline is composed of an image validation layer, a YOLOv8 object detector for crop verification, a Vision Transformer (ViT) classification model for leaf disease identification, and an image hashing duplicate detector.

Computer Vision Pipeline Layers:

- **Image Quality Checker:** Analyzes uploads via OpenCV metrics: brightness (HSV mean), contrast (grayscale std deviation), sharpness (Laplacian variance), resolution (pixel count), and visibility (histogram distribution). Any upload failing checks receives a Warning badge.
- **Duplicate Verification:** Computes and stores SHA-256 hashes of crop images. If a farmer uploads an identical image, it triggers a warning in the Admin dashboard indicating duplicate registration fraud.
- **YOLOv8 Crop Detector:** Locates crop objects and verifies if the crop name entered by the farmer matches the visual content. If the farmer claims Wheat but the image shows Tomatoes, the batch is flagged as NOT_AUTHENTICATED.
- **ViT Leaf Disease Diagnostics:** Analyzes texture patterns and brown/yellow spot ratios to grade leaf health. It compiles organic and chemical treatment instructions and returns them to the farmer.

4. System Workflows & Comprehensive Test Manual

This section serves as the step-by-step operating reference for Quality Assurance teams to verify the AgriTrace supply chain constraints under both normal and edge cases.

Scenario	Normal Case Validation	Edge Case Validation
Farmer Coordinates Validation	Register a farmer with coordinates in range (e.g., 13.13,78.13). The profile registers successfully.	Enter coordinates containing text or out of range (e.g., 91.0,185.0). The server must reject registration with 'wrong location' error (HTTP 400).
Dual Recovery Forgot Password	Enter correct email, receive OTP on mail, enter correct OTP, password resets successfully.	Fail OTP, click 'Try another way'. Present registered security question. Enter matching answer to reset password.
AgriShield Seasonal Monitoring	Upload Wheat in Winter season. System registers crop and saves metadata without warning.	Upload Wheat in Summer season. System triggers Warning and blocks batch until farmer submits an explanation.
Automated Suspension Block	Farmer fails to write explanation within deadline. System automatically triggers suspension.	Suspended farmer logs in. All tabs are disabled, and only the Support Chat channel remains accessible.
Leaflet Privacy Map Overlays	Farmer and Admin view map. Display exact boundary surveyor polygon.	Buyer views map. Exact coordinates are filtered from API. Display fuzzed village overlay circle.

Telegram QR Scan Bot	Upload photo of QR code to bot. Bot decodes QR, queries ledger database, and replies with crop info.	Send 'verify AGR-INVALID'. Bot handles gracefully and returns 'NOT_AUTHENTICATED' warning report.

5. Installation & Operating Reference

Follow these steps to deploy and run the system on a clean environment:

1. Initialize PostgreSQL Database:

Run pgAdmin, create a database named 'agritrace_db', open the Query Tool, paste the contents of 'agritrace_database_schema.sql', and execute (F5).

2. Configure Environment Variables:

Create 'backend/.env' containing your database credentials, SMTP credentials, OpenRouter API key, and Telegram Bot token.

3. Run Backend Server:

Navigate to backend directory, activate python virtual environment, run 'pip install -r requirements.txt', then execute 'python run.py'.

4. Run Frontend Server:

Navigate to frontend directory, execute 'npm install', then execute 'npm run dev'. Access the app at http://localhost:5173.

5. Run Automated Tests:

Execute pytest validation from the project root using the following command:

```
$env:PYTHONPATH="backend"; pytest tests/e2e/test_auth.py
```